LAPORAN PRAKTIKUM PERTEMUAN 3
PEMROGRAMAN MOBILE
FORM INPUT DAN STATE MANAGEMENT



Disusun oleh:
Angga Aditya Nugraha (20240801165)

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS ESA UNGGUL
TANGERANG
2026

Pelaksanaan Praktikum

Bagian A - Input Dasar dengan TextField

```
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 3',
      theme: ThemeData(colorSchemeSeed: Colors.blue, useMaterial3: true),
      home: const InputPage(),
    );
  }
}

class InputPage extends StatefulWidget {
  const InputPage({super.key});

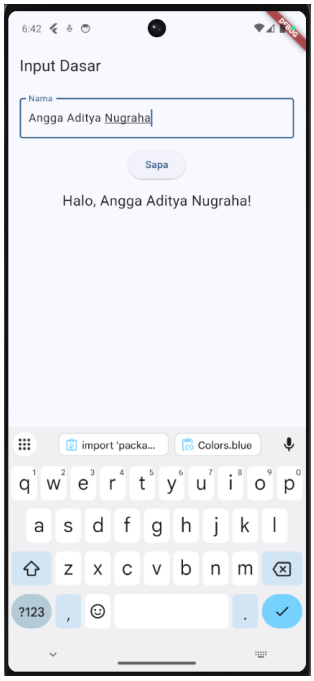
  @override
  State<InputPage> createState() => _InputPageState();
}

class _InputPageState extends State<InputPage> {
  final _controller = TextEditingController();
  String _salam = '';

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Input Dasar')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: _controller,
              decoration: const InputDecoration(
                labelText: 'Nama',
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 12),
            ElevatedButton(
              onPressed: () {
                setState(() => _salam = 'Halo, ${_controller.text}!');
              },
              child: const Text('Sapa'),
            ),
            const SizedBox(height: 12),
            Text(_salam, style: const TextStyle(fontSize: 20)),
          ],
        ),
      ),
    );
  }
```

```
}
```



Sintaks	Penjelasan Fungsi
`TextEditingController`	Objek yang menyimpan dan membaca teks yang diketik pengguna di dalam TextField.
`TextField`	Widget kolom input teks; nilainya diambil lewat properti controller.
`dispose()`	Method yang dipanggil saat widget dihancurkan; controller wajib dilepas (dispose) di sini agar tidak terjadi kebocoran memori.
`setState(() => _salam = ...)`	Memperbarui variabel _salam sekaligus memberi tahu Flutter agar tampilan dibangun ulang.

Bagian B - Form dengan Validasi

```
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 3',
      theme: ThemeData(colorSchemeSeed: Colors.blue, useMaterial3: true),
      home: const FormPage(),
    );
  }
}

class FormPage extends StatefulWidget {
  const FormPage({super.key});

  @override
```

```
  State<FormPage> createState() => _FormPageState();
}

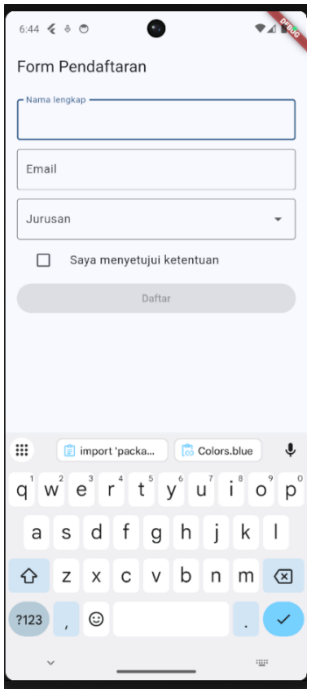
class _FormPageState extends State<FormPage> {
  final _formKey = GlobalKey<FormState>();
  final _nama = TextEditingController();
  final _email = TextEditingController();
  String? _jurusan;
  bool _setuju = false;

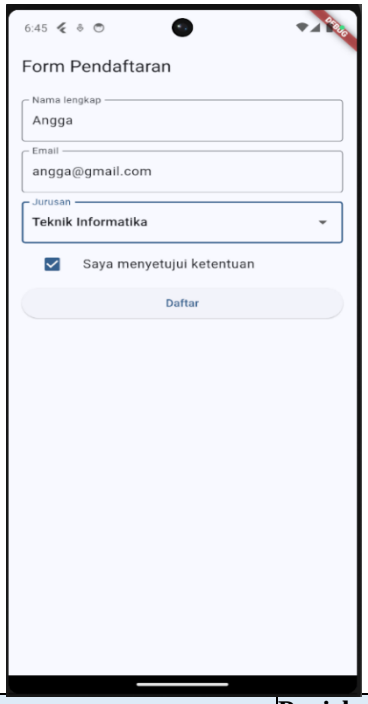
  @override
  void dispose() {
    _nama.dispose();
    _email.dispose();
    super.dispose();
  }

  void _kirim() {
    if (_formKey.currentState!.validate()) {
      final jurusan = _jurusan ?? '-';
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Terdaftar: ${_nama.text} ($jurusan)')),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Form Pendaftaran')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: _nama,
              decoration: const InputDecoration(
                labelText: 'Nama lengkap',
                border: OutlineInputBorder(),
              ),
              validator: (v) =>
                  (v == null || v.trim().isEmpty) ? 'Nama wajib diisi' : null,
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _email,
              keyboardType: TextInputType.emailAddress,
              decoration: const InputDecoration(
                labelText: 'Email',
                border: OutlineInputBorder(),
              ),
              validator: (v) {
                if (v == null || !v.contains('@')) return 'Email tidak valid';
                return null;
              },
            ),
            const SizedBox(height: 12),
            DropdownButtonFormField<String>(
              decoration: const InputDecoration(
                labelText: 'Jurusan',
                border: OutlineInputBorder(),
              ),
              items: const [
                DropdownMenuItem(value: 'TI', child: Text('Teknik Informatika')),
                DropdownMenuItem(value: 'SI', child: Text('Sistem Informasi')),
                DropdownMenuItem(value: 'TE', child: Text('Teknik Elektro')),
              ],
```

```
              onChanged: (v) => setState(() => _jurusan = v),
              validator: (v) => v == null ? 'Pilih jurusan' : null,
            ),
            CheckboxListTile(
              title: const Text('Saya menyetujui ketentuan'),
              value: _setuju,
              controlAffinity: ListTileControlAffinity.leading,
              onChanged: (v) => setState(() => _setuju = v ?? false),
            ),
            ElevatedButton(
              onPressed: _setuju ? _kirim : null,
              child: const Text('Daftar'),
            ),
          ],
        ),
      ),
    );
  }
}
```





Sintaks	Penjelasan Fungsi
`GlobalKey<FormState>`	Kunci yang menghubungkan widget Form dengan state-nya, dipakai untuk memanggil validate() dari luar Form.
`Form`	Widget pembungkus yang mengelola validasi semua TextFormField/dropdown di dalamnya sekaligus.
`TextFormField + validator`	Seperti TextField, tetapi memiliki properti validator yang mengembalikan pesan error (String) atau null bila nilainya valid.
`_formKey.currentState!.validate()`	Menjalankan seluruh validator di dalam Form; mengembalikan true hanya jika semua isian valid.
`DropdownButtonFormField`	Dropdown pilihan yang juga mendukung validator, sehingga bisa diwajibkan untuk dipilih.
`ScaffoldMessenger.of(context).showSnackBar()`	Menampilkan notifikasi singkat (SnackBar) di bagian bawah layar.

Bagian C - Mengapa Butuh State Management?

Bagian ini bersifat teori. Pada Bagian A dan B, state (_salam, _nama, dst.) hanya hidup di dalam satu halaman melalui setState. Pada kasus aplikasi dengan dua halaman yang perlu berbagi data yang sama (misalnya halaman daftar tugas dan halaman tambah tugas), mengoper data lewat constructor dan callback antar halaman akan cepat menjadi rumit. Solusinya adalah memindahkan data ke satu objek bersama yang dapat diakses oleh semua halaman terkait, yaitu dengan menggunakan Provider.

Bagian D - Aplikasi Daftar Tugas dengan Provider

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class Tugas {
  String judul;
  bool selesai;
  Tugas(this.judul, {this.selesai = false});
}

class TugasModel extends ChangeNotifier {
  final List<Tugas> _items = [];
  List<Tugas> get items => List.unmodifiable(_items);
  int get jumlahSelesai => _items.where((t) => t.selesai).length;

  void tambah(String judul) {
    _items.add(Tugas(judul));
    notifyListeners();
  }

  void toggle(int index) {
    _items[index].selesai = !_items[index].selesai;
    notifyListeners();
  }

  void hapus(int index) {
    _items.removeAt(index);
    notifyListeners();
  }
}

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => TugasModel(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Daftar Tugas',
      theme: ThemeData(colorSchemeSeed: Colors.blue, useMaterial3: true),
      home: const TugasPage(),
    );
  }
}

class TugasPage extends StatelessWidget {
  const TugasPage({super.key});
```

```
  @override
  Widget build(BuildContext context) {
    final model = context.watch<TugasModel>();
    return Scaffold(
      appBar: AppBar(
        title: Text('Tugas (${model.jumlahSelesai}/${model.items.length})'),
      ),
      body: ListView.builder(
        itemCount: model.items.length,
        itemBuilder: (context, i) {
          final t = model.items[i];
          return ListTile(
            leading: Checkbox(
              value: t.selesai,
              onChanged: (_) => context.read<TugasModel>().toggle(i),
            ),
            title: Text(
              t.judul,
              style: TextStyle(
                decoration: t.selesai ? TextDecoration.lineThrough : null,
              ),
            ),
            trailing: IconButton(
              icon: const Icon(Icons.delete),
              onPressed: () => context.read<TugasModel>().hapus(i),
            ),
          );
        },
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () {
          Navigator.push(
            context,
            MaterialPageRoute(builder: (_) => const TambahPage()),
          );
        },
        child: const Icon(Icons.add),
      ),
    );
  }
}

class TambahPage extends StatefulWidget {
  const TambahPage({super.key});

  @override
  State<TambahPage> createState() => _TambahPageState();
}

class _TambahPageState extends State<TambahPage> {
  final _controller = TextEditingController();

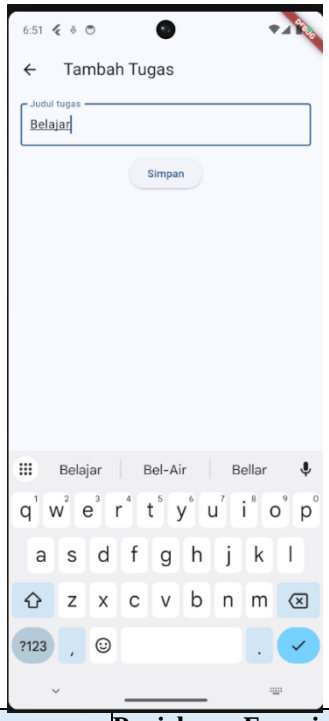
  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  void _simpan() {
    final judul = _controller.text.trim();
    if (judul.isEmpty) return;
    context.read<TugasModel>().tambah(judul);
    Navigator.pop(context);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
```

```
      appBar: AppBar(title: const Text('Tambah Tugas')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: _controller,
              autofocus: true,
              decoration: const InputDecoration(
                labelText: 'Judul tugas',
                border: OutlineInputBorder(),
              ),
              onSubmitted: (_) => _simpan(),
            ),
            const SizedBox(height: 12),
            ElevatedButton(onPressed: _simpan, child: const Text('Simpan')),
          ],
        ),
      ),
    );
  }
}
```







Sintaks	Penjelasan Fungsi
`ChangeNotifier`	Class dasar yang menyediakan mekanisme notifyListeners() untuk memberi tahu widget yang berlangganan bahwa data berubah.
`ChangeNotifierProvider`	Widget yang menempatkan satu instance ChangeNotifier (di sini TugasModel) di atas pohon widget agar dapat diakses oleh semua widget di bawahnya.
`notifyListeners()`	Dipanggil setiap kali data pada model berubah (tambah, toggle, hapus), sehingga widget yang berlangganan dibangun ulang otomatis.
`context.watch<TugasModel>()`	Membaca data model dan membuat widget ikut dibangun ulang setiap kali notifyListeners() dipanggil; dipakai di dalam build().
`context.read<TugasModel>()`	Membaca data model satu kali tanpa membuat widget dibangun ulang; dipakai di dalam callback seperti onPressed atau onChanged.

3. Latihan Mandiri

Empat soal dikerjakan:

(1) validasi judul minimal 3 karakter di TambahPage,

(2) method hapusSelesai() beserta tombolnya di AppBar,

(3) SnackBar konfirmasi setelah tugas disimpan,

(4) teks "Belum ada tugas" saat daftar kosong.

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class Tugas {
  String judul;
  bool selesai;
  Tugas(this.judul, {this.selesai = false});
}

class TugasModel extends ChangeNotifier {
  final List<Tugas> _items = [];
  List<Tugas> get items => List.unmodifiable(_items);
  int get jumlahSelesai => _items.where((t) => t.selesai).length;

  void tambah(String judul) {
    _items.add(Tugas(judul));
    notifyListeners();
  }

  void toggle(int index) {
    _items[index].selesai = !_items[index].selesai;
    notifyListeners();
  }

  void hapus(int index) {
    _items.removeAt(index);
    notifyListeners();
  }

  // Soal 2: hapus semua tugas yang sudah selesai
  void hapusSelesai() {
    _items.removeWhere((t) => t.selesai);
    notifyListeners();
  }
}

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => TugasModel(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Daftar Tugas',
      theme: ThemeData(colorSchemeSeed: Colors.blue, useMaterial3: true),
      home: const TugasPage(),
    );
  }
}
```

```
class TugasPage extends StatelessWidget {
  const TugasPage({super.key});

  @override
  Widget build(BuildContext context) {
    final model = context.watch<TugasModel>();
    return Scaffold(
      appBar: AppBar(
        title: Text('Tugas (${model.jumlahSelesai}/${model.items.length})'),
        actions: [
          // Soal 2: tombol hapus tugas selesai
          IconButton(
            icon: const Icon(Icons.playlist_remove),
            tooltip: 'Hapus yang selesai',
            onPressed: () => context.read<TugasModel>().hapusSelesai(),
          ),
        ],
      ),
      // Soal 4: tampilkan teks kalau daftar kosong
      body: model.items.isEmpty
          ? const Center(child: Text('Belum ada tugas'))
          : ListView.builder(
              itemCount: model.items.length,
              itemBuilder: (context, i) {
                final t = model.items[i];
                return ListTile(
                  leading: Checkbox(
                    value: t.selesai,
                    onChanged: (_) => context.read<TugasModel>().toggle(i),
                  ),
                  title: Text(
                    t.judul,
                    style: TextStyle(
                      decoration: t.selesai ? TextDecoration.lineThrough : null,
                    ),
                  ),
                  trailing: IconButton(
                    icon: const Icon(Icons.delete),
                    onPressed: () => context.read<TugasModel>().hapus(i),
                  ),
                );
              },
            ),
      floatingActionButton: FloatingActionButton(
        onPressed: () {
          Navigator.push(
            context,
            MaterialPageRoute(builder: (_) => const TambahPage()),
          );
        },
        child: const Icon(Icons.add),
      ),
    );
  }
}

class TambahPage extends StatefulWidget {
  const TambahPage({super.key});

  @override
  State<TambahPage> createState() => _TambahPageState();
}

class _TambahPageState extends State<TambahPage> {
  // Soal 1: ganti TextField jadi Form + TextFormField dengan validasi
  final _formKey = GlobalKey<FormState>();
  final _controller = TextEditingController();
```

```
  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  void _simpan() {
    if (_formKey.currentState!.validate()) {
      context.read<TugasModel>().tambah(_controller.text.trim());
      // Soal 3: SnackBar konfirmasi, ditampilkan sebelum pop
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Tugas ditambahkan')),
      );
      Navigator.pop(context);
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tambah Tugas')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Form(
          key: _formKey,
          child: Column(
            children: [
              TextFormField(
                controller: _controller,
                autofocus: true,
                decoration: const InputDecoration(
                  labelText: 'Judul tugas',
                  border: OutlineInputBorder(),
                ),
                validator: (v) {
                  if (v == null || v.trim().length < 3) {
                    return 'Judul minimal 3 karakter';
                  }
                  return null;
                },
                onFieldSubmitted: (_) => _simpan(),
              ),
              const SizedBox(height: 12),
              ElevatedButton(onPressed: _simpan, child: const Text('Simpan')),
            ],
          ),
        ),
      ),
    );
  }
}
```



4. Tugas: Daftar Belanja

State barang belanja disimpan dalam satu ChangeNotifier (BelanjaModel) dan dibagikan lewat Provider. Form Tambah memvalidasi nama (wajib), jumlah (wajib, harus angka lebih dari 0), dan kategori (wajib dipilih):

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class Barang {
  String nama;
  int jumlah;
  String kategori;
  bool dibeli;
  Barang(this.nama, this.jumlah, this.kategori, {this.dibeli = false});
}

class BelanjaModel extends ChangeNotifier {
  final List<Barang> _items = [];
  List<Barang> get items => List.unmodifiable(_items);
  int get belumDibeli => _items.where((b) => !b.dibeli).length;

  void tambah(String nama, int jumlah, String kategori) {
    _items.add(Barang(nama, jumlah, kategori));
    notifyListeners();
  }

  void toggle(int index) {
    _items[index].dibeli = !_items[index].dibeli;
    notifyListeners();
  }

  void hapus(int index) {
    _items.removeAt(index);
    notifyListeners();
  }
}
```

```
void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => BelanjaModel(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Daftar Belanja',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.blue, useMaterial3: true),
      home: const BelanjaPage(),
    );
  }
}

class BelanjaPage extends StatelessWidget {
  const BelanjaPage({super.key});

  @override
  Widget build(BuildContext context) {
    final model = context.watch<BelanjaModel>();
    return Scaffold(
      appBar: AppBar(
        title: Text('Belanja (${model.belumDibeli} belum dibeli)'),
      ),
      body: model.items.isEmpty
          ? const Center(child: Text('Belum ada barang'))
          : ListView.builder(
              itemCount: model.items.length,
              itemBuilder: (context, i) {
                final b = model.items[i];
                return ListTile(
                  leading: Checkbox(
                    value: b.dibeli,
                    onChanged: (_) => context.read<BelanjaModel>().toggle(i),
                  ),
                  title: Text(
                    '${b.nama} (${b.jumlah})',
                    style: TextStyle(
                      decoration: b.dibeli ? TextDecoration.lineThrough : null,
                    ),
                  ),
                  subtitle: Text(b.kategori),
                  trailing: IconButton(
                    icon: const Icon(Icons.delete),
                    onPressed: () => context.read<BelanjaModel>().hapus(i),
                  ),
                );
              },
            ),
      floatingActionButton: FloatingActionButton(
        onPressed: () {
          Navigator.push(
            context,
            MaterialPageRoute(builder: (_) => const TambahBarangPage()),
          );
        },
        child: const Icon(Icons.add),
      ),
    );
```

```
  }
}

class TambahBarangPage extends StatefulWidget {
  const TambahBarangPage({super.key});

  @override
  State<TambahBarangPage> createState() => _TambahBarangPageState();
}

class _TambahBarangPageState extends State<TambahBarangPage> {
  final _formKey = GlobalKey<FormState>();
  final _nama = TextEditingController();
  final _jumlah = TextEditingController();
  String? _kategori;

  @override
  void dispose() {
    _nama.dispose();
    _jumlah.dispose();
    super.dispose();
  }

  void _simpan() {
    if (_formKey.currentState!.validate()) {
      context.read<BelanjaModel>().tambah(
            _nama.text.trim(),
            int.parse(_jumlah.text.trim()),
            _kategori!,
          );
      Navigator.pop(context);
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tambah Barang')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: _nama,
              decoration: const InputDecoration(
                labelText: 'Nama barang',
                border: OutlineInputBorder(),
              ),
              validator: (v) =>
                  (v == null || v.trim().isEmpty) ? 'Nama wajib diisi' : null,
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _jumlah,
              keyboardType: TextInputType.number,
              decoration: const InputDecoration(
                labelText: 'Jumlah',
                border: OutlineInputBorder(),
              ),
              validator: (v) {
                if (v == null || v.trim().isEmpty) return 'Jumlah wajib diisi';
                final n = int.tryParse(v.trim());
                if (n == null || n <= 0) return 'Jumlah harus angka lebih dari 0';
                return null;
              },
            ),
            const SizedBox(height: 12),
```

```
            DropdownButtonFormField<String>(
              decoration: const InputDecoration(
                labelText: 'Kategori',
                border: OutlineInputBorder(),
              ),
              items: const [
                DropdownMenuItem(value: 'Makanan', child: Text('Makanan')),
                DropdownMenuItem(value: 'Minuman', child: Text('Minuman')),
                DropdownMenuItem(value: 'Kebutuhan Rumah', child: Text('Kebutuhan Rumah')),
                DropdownMenuItem(value: 'Lainnya', child: Text('Lainnya')),
              ],
              onChanged: (v) => setState(() => _kategori = v),
              validator: (v) => v == null ? 'Pilih kategori' : null,
            ),
            const SizedBox(height: 20),
            ElevatedButton(onPressed: _simpan, child: const Text('Simpan')),
          ],
        ),
      ),
    );
  }
}
```

- **CheckboxListTile** — gabungan Checkbox dan ListTile dalam satu baris, lengkap dengan label teks yang bisa ikut diketuk.
- **keyboardType: TextInputType.emailAddress / TextInputType.**number — mengatur jenis keyboard yang muncul agar sesuai jenis input (email, angka).
- **String? _jurusan;** — tanda ? menunjukkan variabel ini boleh bernilai null (belum dipilih).
- **bool _setuju = false;** — variabel bertipe boolean (benar/salah) sebagai status checkbox.
- **_setuju ? _kirim : null** — operator ternary; kalau _setuju true jalankan _kirim, kalau false nilainya null (yang membuat tombol nonaktif).
- **v == null || v.trim().isEmpty** — operator || (atau); .trim() menghapus spasi di awal/akhir teks, .isEmpty mengecek apakah string kosong.
- **List<Tugas> get items => List.unmodifiable(_items**); — getter, properti yang dihitung lewat fungsi; List.unmodifiable mengembalikan salinan list yang tidak bisa diubah dari luar class.
- **int get jumlahSelesai => _items.where((t) => t.selesai).length;** — where() menyaring list sesuai kondisi (lambda (t) => t.selesai), .length menghitung jumlah hasilnya.
- **_items.removeWhere((t) => t.selesai);** — menghapus semua elemen list yang memenuhi kondisi.
- **MaterialPageRoute(builder: (_) => const TambahPage())** — builder adalah fungsi yang mengembalikan widget halaman tujuan; _ artinya parameter context di situ tidak dipakai.
- **int.tryParse(v)** — mencoba mengubah teks jadi angka; mengembalikan null kalau teksnya bukan angka (lebih aman dari int.parse yang bisa menyebabkan error).
- **TextDecoration.lineThrough** — gaya teks yang dicoret.
- **autofocus: true** — otomatis memunculkan keyboard dan fokus ke kolom ini saat halaman dibuka.
- **onSubmitted / onFieldSubmitted** — callback yang dijalankan saat pengguna menekan tombol "selesai/enter" di keyboard.

5. Jawaban Pertanyaan Refleksi

1. Mengapa TextEditingController harus di-dispose?

TextEditingController mengalokasikan resource (listener dan buffer teks) yang tetap ada di memori selama controller tidak dilepas. Jika halaman yang memakainya ditutup tanpa memanggil dispose(), resource tersebut tidak dibersihkan sehingga terjadi kebocoran memori (memory leak), apalagi jika halaman tersebut dibuka-tutup berulang kali.

2. Kapan cukup memakai setState, dan kapan sebaiknya beralih ke Provider?

setState cukup dipakai ketika data hanya memengaruhi tampilan satu widget/halaman saja (state lokal/ephemeral), misalnya status tercentang pada satu checkbox. Provider sebaiknya dipakai ketika data perlu dibagikan dan tetap sinkron di beberapa widget atau halaman sekaligus (app state), seperti daftar tugas atau daftar belanja yang diakses dari halaman daftar maupun halaman tambah.

3. Apa yang terjadi bila notifyListeners() lupa dipanggil? Mengapa?

Data di dalam model akan tetap berubah sesuai logika kode, tetapi tampilan di layar tidak akan diperbarui dan terlihat seperti tidak terjadi apa-apa. Ini karena widget yang memakai context.watch() hanya dibangun ulang ketika menerima notifikasi dari notifyListeners(); tanpa pemanggilan tersebut, Flutter tidak mengetahui bahwa ada perubahan data yang perlu ditampilkan.

4. Mengapa di dalam onPressed kita memakai context.read, bukan context.watch?

context.watch menandai widget agar ikut dibangun ulang setiap kali data berubah, sehingga penggunaannya seharusnya di dalam method build(). Di dalam callback seperti onPressed, kita hanya perlu mengakses model sekali untuk memanggil method (misalnya tambah atau hapus), tanpa perlu widget tersebut dibangun ulang pada saat itu juga, sehingga context.read lebih tepat dan lebih efisien digunakan di sana.